

MyScoutee Engineering Deep Dive

Architecture / delivery methods / decisions

A cross-repository guide to the MyScoutee architecture, including Firebase configuration, delivery methods and technology choices.

Contents

1. Context and ownership

2. Application and data architecture

3. Deployment and operations

4. Security and integrations

5. Evidence, decisions and change control

6. Architecture approaches applied to MyScoutee

7. Firebase in the architecture

Software delivery methods: from sequential plans to agile delivery

Marketing decisions and product evidence

Cost and efficiency decisions

Why Netcup was simpler first: cloud and edge cost anatomy

Comparable cloud stacks and billing: AWS, Azure, Google Cloud and OCI

Technology trials and integration status

1. Context and ownership

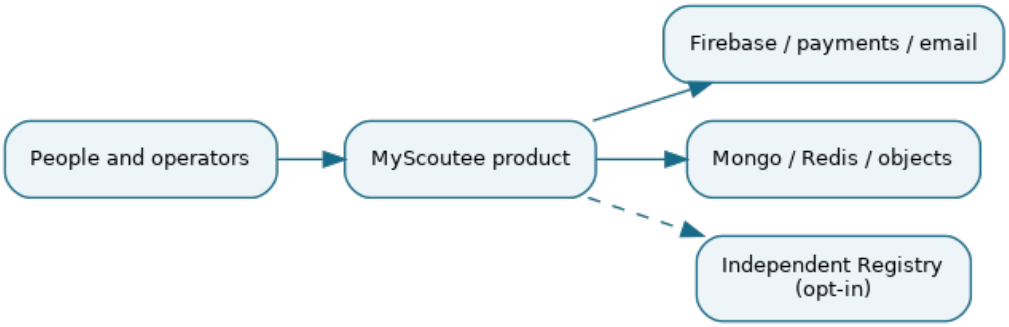
Status: current-source architecture, with experimental boundaries called out explicitly. See the source ledger.

Purpose and actors

MyScoutee organizes Dating, Work and Community activity around a completed, mutually welcomed real-world meeting. The product supports profiles, mutual priorities, events and groups, work campaigns and service offerings, community cases, announcements, voting, chat, calls, notifications and human ratings. API and MCP clients may perform permitted organizing tasks; personal decisions such as ratings and ballots remain in the human UI.

Actor	Main interaction	Authority boundary
Member or group profile	Explore, organize, participate and converse	Own session and profile/group permissions
Administrator	Moderation, configuration and scoped statistics	Administrator role, separate from Operator
Deployment Operator	Deployment configuration, updates and Registry enrollment	Authenticated Operator; host policy still limits Registry egress
External client	Profile-bound integration API or MCP	Claimed token/connection, pack and existing application permissions
Registry operator	Independent operator network and protocol	Separate deployment, identity and database

System context



These external connections are conditional. A bootstrap Operator login exists independently of Firebase; Firebase activation follows staging and checks. A payment simulator and Explore Registry serve demo/test paths. Neither represents a production payment provider or the Operator-selected Registry.

Canonical owners

Repository	Responsibility
Backend	Product Java domains, composition inputs, packaging selections, QA and release evidence
Frontend	Angular UI, HTTP/local adapters and user workflows; recorded as backend submodule
Base	Published shared Java business libraries and BOM for backend and template
Components	Shared UI and technical components
Microservices	Reusable assembly, KVM/K3s, Helm, transport and monitoring mechanisms
Registry	Independent Go Registry server, protocol and SQLite persistence
nginx, MCP	Shared edge and MCP implementations consumed as versioned images
Packaging	Shared installer, update and recovery tooling
Template	Separate application using shared dependencies, with its own product and deployment configuration

The architecture repository is a derived map. It must not become a second owner of API schemas, business code, secrets, image locks or deployment configuration.

Architectural principles visible in the source

1. Product domain implementations have one backend owner. The eight-service lab selects those same source units rather than duplicating them. Cross-service calls use API/DTO contracts and explicit transport bindings.

2. The source/packaged Compose application and KVM microservices lab are different runtime forms. A build does not establish distributed runtime qualification.

3. The Registry is optional and independent. Its identity and receipt boundary is not the product data store.

4. Release notes, QA records and KVM checks are separate evidence sets; a lab feature is not automatically a product feature.

2. Application and data architecture

Status: current source and configured runtime paths. The monolith and distributed lab are distinct deployment views. See source ledger.

Request and adapter path

Angular selects `local` or `http` adapters through environment and route configuration. Development, E2E and production select HTTP with `/api`; default/GitHub examples select local. The local path uses browser-side sources, seed builders and IndexedDB and cannot prove server behavior. In HTTP mode nginx serves the frontend and routes API/MCP traffic to the Java application and MCP gateway. Angular initialization loads translations and session state; its HTTP chain handles connectivity, session mode, authentication and payment-provider synchronization.



Exact endpoints, session headers and write preconditions belong to owner contracts and API guides. The diagram shows a representative path, not every route.

REST and SOAP at the API boundary

REST is an architectural style that organizes HTTP APIs around resources and methods such as GET and P0ST; JSON is a common payload format. SOAP is an operation-oriented messaging protocol that wraps XML messages in a SOAP envelope, commonly with a WSDL contract. SOAP can use HTTP, but that alone does not make it REST.

MyScoutee's browser and external integration paths use HTTP endpoints with JSON payloads (and multipart uploads where needed). Its split-service calls use typed HTTP/JSON contracts described by OpenAPI; live chat uses WebSocket. The inspected product baseline has no SOAP endpoint or WSDL. Connecting a SOAP-only partner would require an explicit adapter that preserves MyScoutee's existing authorization and data boundaries.

Java source and assembly

The product backend has one Java application project. Its domains cover accounts, access, administration, affinity, assets, campaigns, chat, community, content, email, events, feedback, groups, help, integrations, localization, media, notifications, operator functions, payments, platform functions and registry integration. Shared code covers configuration, security, delivery and Mongo support. The KVM lab assembles this same source into eight services; it is another deployment form, not a second product implementation.

KVM assembly	Domain groups selected from the same Java source
`identity`	accounts, access, administration, platform, integrations
`community`	groups, community, events, campaigns, content, feedback
`communication`	chat, notifications, email, localization
`assets`	assets, media
`payments`	payments
`affinity`	affinity
`help`	help
`network`	operator, registry

Assembly configuration defines the eight groups and their budgets. Explicit transport bindings define implemented cross-service contracts; the gateway maps public routes and keeps internal and repository routes private. The lab retains REAL and DEMO databases in one Mongo replica set. The split does not create eight independently owned data products.

Persistence boundaries

Store	Current role	Critical limit
MongoDB	Product documents, configuration, jobs, outboxes, REAL/DEMO databases	Source Compose defaults to a single-member replica set for change streams; it does not provide database failover
Redis	Cache and coordination used by selected domains	Not canonical business record
MinIO/object store	Media object bytes	Object and Mongo metadata require coherent recovery
Browser IndexedDB	Local adapter and selected browser caches/outboxes	Does not prove server persistence or authorization
Registry SQLite	Independent Registry protocol state	Explore Registry state must not mix with an external Registry

DEMO data is disposable. The single-member Mongo replica set enables change streams and retains the official initialization path.

Why MongoDB?

Profiles, groups and events in MyScoutee contain optional sections and lists that vary between records. MongoDB stores each record with its own details in a document, making that record the unit of an atomic write. A normalized relational design would spread those details across related tables and require coordinated row changes for the same update. This is why the product uses MongoDB for its data; it has no parallel relational store.

Example: affinity recomputation

A profile, event, asset or configuration change enqueues a durable affinity task with a key, target, status and availability time. The worker claims due work and calculates projections from product data. Normal Compose configuration uses Mongo change streams to wake the worker in REAL and DEMO; reconnect scans pending work and timers handle future deadlines. Fallback polling is normally disabled.

The Big Data lab's Kafka/Flink affinity messages are synthetic; they do not replace this task store, scoring inputs, retries or authorization. Notifications, email and Registry reporting also have durable outbox/worker paths in Java. Their exact delivery contracts remain with their source and tests.

3. Deployment and operations

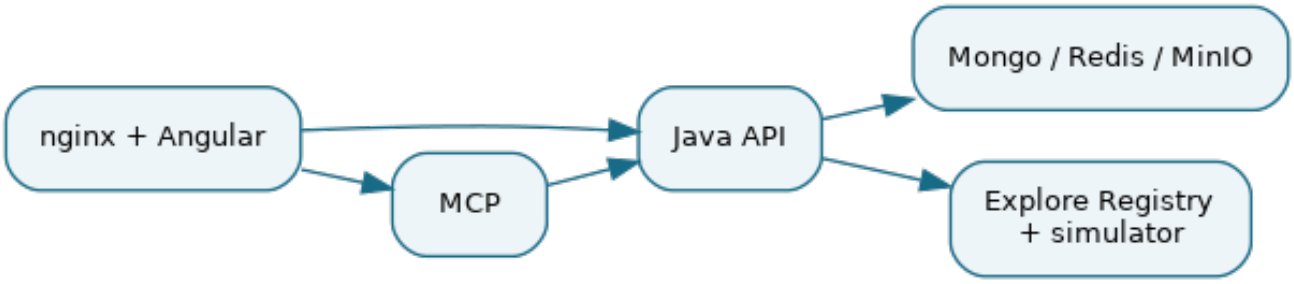
Status: configuration-based comparison. A source stack, an installed release and a KVM lab have different qualification scopes. See source ledger.

Runtime forms

Form	Runtime	Purpose and evidence limit
Backend source development	Development Compose over shared base/replica configuration	HTTP app, frontend sync, local Registry/MCP and simulators; no automatic restart
Backend source E2E	Isolated Compose test stack	Separate test project and fixtures; no automatic restart
Backend source production	Source production Compose model	Production images and `unless-stopped` policy
Installed application	Versioned Debian package and Compose runtime	Installed product on Netcup; qualified separately from source stacks and the KVM lab
KVM microservices lab	Eight Java assemblies on K3s/Helm	Lab monitoring; not the installed Compose product

The source Compose models select their own Registry/MCP image variants and separate their demo data/key state. Their legacy published ports and fixed service names mean dev/E2E/prod stacks are not generally concurrent on one Docker host.

Container topology



The independent external Registry and real payment providers sit outside this deployment. The source production and packaged models each have their own concrete image and volume selections. This diagram is logical; inspect the Compose file for exact ports, mounts, TLS files and health checks before operation.

KVM split path

The product has eight service groups in the KVM lab. Its shared runtime builds images from canonical product source and resolved dependency inputs, reuses unchanged image identities, renders Helm assets and manages the KVM/K3s deployment. Product runtime configuration sets the namespace, database names, dependencies, frontend and metrics NodePorts, and monitoring project. The gateway routes by explicit public path while internal APIs stay private. Mongo uses a replica set in the lab.

The lab's `mongo-seed` image volume holds initialization scripts and canonical REAL/DEMO seed inputs; data persists on separate PVCs. A changed seed artifact does not reinitialize an existing Mongo volume. Lab Prometheus/Grafana monitoring is distinct from the standalone Elastic lab.

State, failure and recovery boundaries

Boundary	Consequence
Single-member Compose Mongo	Change streams work, but node loss interrupts writes; no automatic Mongo failover
Mongo plus MinIO	Domain metadata and object bytes can diverge after partial restore
Registry deployment identity	Private key, state and receipts must remain coherent; loss is not a silent re-enrollment
Explore Registry	Isolated demo SQLite/key/Java-client state; never a recovery source for external Registry
Firebase activation	Staged configuration requires authentication and notification checks before activation
Installer/upgrade	Versioned image and migration inputs require the packaging owner's qualification workflow

This guide does not issue backup or restore commands. The current operator/packaging manuals own them. An architecture diagram cannot establish recovery point, recovery time or successful host restoration without a matched restore test.

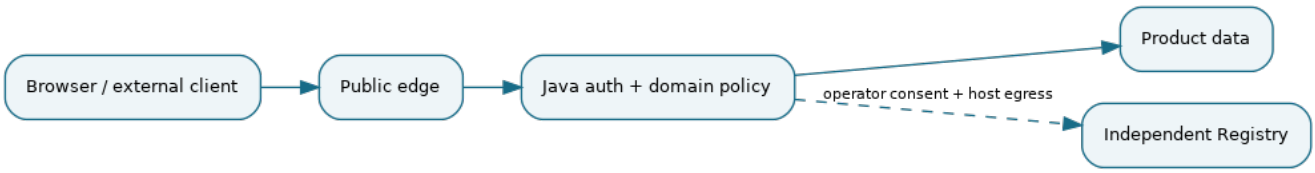
Observability and verification

The backend has runtime health, QA records and release evidence. The KVM lab exposes metrics through a restricted guest-only gateway listener and configured Prometheus/Grafana. The separate Elastic repository is a local source-search/observability lab, not the product telemetry plane. Any future production observability decision must specify sources, redaction, retention, access, costs and alert ownership, then verify the path end to end.

4. Security and integrations

Status: inspected source boundaries, not a penetration-test result or security certification. See source ledger.

Trust boundaries



Java authentication distinguishes sessions, integration tokens and admin clients. Route policy scopes integration, Operator and media access; method and domain checks enforce the permissions for individual actions. A route's presence in public configuration is not proof that its mutation is public.

Identity and API permissions

- Operator and Administrator are distinct authorities. A package can create a deployment-local bootstrap Operator credential on first installation. Firebase settings are staged and activated only after checks; Firebase does not automatically revoke the bootstrap login.
- Each integration token represents one external client slot, is claimed by a client identity and can be revoked. API packs limit operations but cannot grant a role the profile lacks.
- Anonymous `/mcp` help and authenticated `/mcp/private` personal operations are separate surfaces. Personal decisions, including ratings and ballots, are not integration writes. A client's advertised operation set is contextual and can change.
- DEMO and REAL have separate Mongo contexts. Demo accounts and isolated service data are not a substitute for checking production roles, persistence or outbound controls.

Registry boundary

The Registry is an independent Go/SQLite deployment that accepts compatible applications without owning their product data. The backend is the only Registry network client in its deployment. A production installation starts uninitialized; the Operator chooses an endpoint, reviews its signed public identity, then confirms enrollment. Host-level `REGISTRY_OUTBOUND_ALLOWED` still gates egress. The deployment identity is Ed25519, with durable state and receipts; private-key loss after enrollment is an explicit recovery problem. The internal Explore Registry uses separate keys, data and client state.

Adjacent repositories are experiments, not connected product services

Repository	Verified local capability	Product boundary
Big Data	Synthetic Kafka stream, ClickHouse queries, Spark/Parquet lake, Flink affinity rollups	Does not read current product Mongo/Redis or change product statistics
Elastic	CodeForge source search, findings and Elastic telemetry; optional Fleet lab	Local evaluation, not the production search/SIEM or MyScoutee telemetry plane
2FA	Standalone Go TOTP/HOTP, WebAuthn and recovery-code API; optional Keycloak lab	No MyScoutee login integration in that repository
Security	Isolated source/image/passive-web scans	Reports require review; no application mutation or certification
Infra	Disposable Ansible/Puppet comparison on a synthetic nginx health page	Does not provision the backend or production edge

A future integration requires an explicit product owner, threat/data-flow analysis, schema and auth contract, retention and operations design, a rollback route and matched runtime evidence. This is a decision checklist, not authorization to connect the labs now.

5. Evidence, decisions and change control

Status: architecture assessment as of the source ledger, not a roadmap commitment.

Verified versus proposed

Claim	Evidence level	Practical consequence
Backend source has one product Java source owner and eight configured KVM assemblies	Configuration and shared runtime README	Inspect/build/test each runtime form independently
Backend source Compose defaults to single-member Mongo replica set	Packaging reference and Compose configuration	Change streams are available; database failover is not established
Installed Compose product has its own deployment evidence	Release and installation records	Do not apply that qualification to source stacks or the KVM lab
Big Data, Elastic, 2FA, security and infra labs work in isolation	Their own READMEs	No MyScoutee integration is implied
A RUP-to-agile path could guide future delivery	Methods comparison and external process sources	A proposal, not evidence that MyScoutee uses RUP, OpenUP or Scrum

Architecture decisions to record when changes are proposed

Decision	Question to answer before implementation
Split-runtime promotion	Which domain/API contracts, data ownership rules, rollback and monolith/KVM checks make a release acceptable?
Product observability	Which events/metrics/logs, privacy controls and response ownership justify a telemetry pipeline?
Big Data adoption	Which product question exceeds current Mongo/Redis/statistics capability, and which source of truth feeds it?
2FA integration	Which login and recovery policy owns factor enrollment and enforcement, and how is the trusted backend-to-factor API secured?
Registry participation	Which operator consent, host egress, key/receipt backup and independent endpoint policy apply?
Recovery objective	What RPO/RTO is required, and what tested restore proves it across Mongo, objects and Registry identity?

These are decision prompts, not planned features. The source owners and their QA/release workflows remain authoritative. This repository should link accepted decisions and update its diagrams only after implementation or an explicitly adopted target architecture.

Update rule

For each material change: (1) identify the canonical owner and revision; (2) update the relevant source ledger row and status; (3) adjust diagrams, data paths and trust boundaries; (4) distinguish configuration, runtime verification and published-release evidence; (5) regenerate and visually check the PDF. Do not copy private credentials, scanner reports or live user data into this repository.

6. Architecture approaches applied to MyScoutee

Status: illustrative views of the inspected MyScoutee baseline, not a claim that the organization adopted TOGAF, C4 or arc42. A proposed change is labeled separately from current source or release evidence. The shared discussion raised TOGAF ADM, C4, arc42 and Architecture Decision Records (ADRs); they answer different questions and can be used together.

No separate architecture application is required to use these examples. This Markdown file contains the editable comparison and Mermaid diagram; the repository generator renders the comparison and a matching diagram in the English PDF. C4, arc42, TOGAF ADM and ADRs name a diagram model, a documentation template, an architecture method and a decision-record format, respectively.

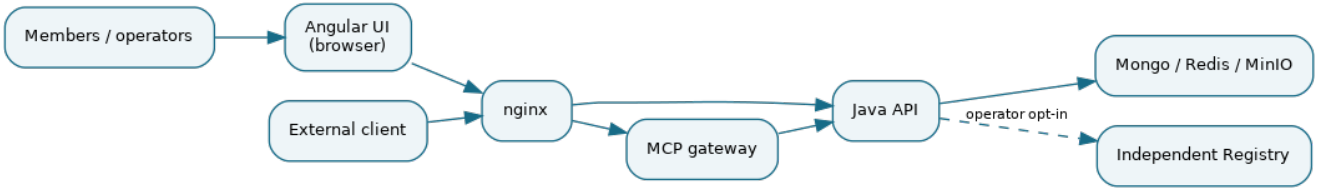
Four approaches, one system

Approach	What it organizes	MyScoutee example	What it cannot prove
C4 model	Diagrams that zoom from people and systems to containers, components and optional code	Show the product boundary, Angular/nginx/Java/MCP/data-store containers, then the affinity task path inside Java	A diagram does not establish that a deployment or recovery path passed qualification
arc42	A tailored architecture document with goals, constraints, structure, runtime, deployment, decisions and risks	Put the existing context, data, security, deployment and evidence chapters under a common reading structure	A filled template is not implementation evidence
TOGAF ADM	An iterative enterprise architecture method connecting business intent, application/data/technology baselines, options, migration and governance	Evaluate whether the Compose product should ever promote a KVM split, with cost, ownership, data and rollback gates	A TOGAF phase name does not turn a lab into an adopted target architecture
ADR	A short record of one architecturally significant decision and its trade-offs	Record the chosen runtime form, alternatives, consequences and evidence when an owner actually decides	A proposed ADR does not authorize or implement its option

Sources: C4 diagrams, arc42 overview, The Open Group's TOGAF overview and ADR templates. These approaches are complementary, not competing deployment technologies.

C4: zoom into the installed Compose product

This is a C4-style container view, not a new runtime design. A C4 container is an application or data store with a runtime responsibility; it is not necessarily a Docker container. The Angular application runs in the browser and is served by nginx. The diagram omits optional payment/email connections and the separate KVM lab to keep one deployment form legible. See context, application/data and deployment for source-backed detail.



C4 zoom	What the MyScoutee view would contain	Evidence boundary
1. System context	Members, Operators, external clients, optional providers and the independent Registry around MyScoutee	The context chapter already provides this view
2. Containers	Angular UI, nginx, Java API, MCP gateway, MongoDB, Redis and object storage	The diagram above describes the installed Compose model logically; exact images, ports and volumes belong to packaging
3. Components	Within Java's affinity area: task service, durable task store, change-stream wake-up, worker and projections	The affinity path is a focused candidate component view, not a claim that every Java package is a separate service
4. Code	A task/worker class or interface diagram only when a specific implementation decision needs it	Generate from current source rather than maintaining a manual class catalog

C4 also has deployment and dynamic diagrams. Here the runtime forms would be separate deployment views for source Compose, installed Compose and KVM/K3s; merging them would imply a runtime that does not exist. C4's diagram guidance recommends using only the levels that add value.

arc42: where the same evidence would live

The arc42 template has 12 tailorable sections. This repository already holds much of their content, but does not claim full arc42 conformance. A compact MyScoutee reading map would be:

arc42 sections	MyScoutee content	Current source in this repo
1-4: goals, constraints, context, strategy	Real-world meeting outcome; source ownership; independent Registry; one Java domain source with alternate assemblies	Context and ownership
5-7: building blocks, runtime, deployment	Java domain groups; request and affinity paths; installed Compose versus KVM lab	Application/data, deployment
8-9: crosscutting concepts, decisions	Permissions, REAL/DEMO isolation, external-provider activation; accepted ADRs when they exist	Security/integrations, evidence/decisions
10-12: quality, risks, glossary	Recovery and privacy scenarios, unverified failover or integration claims, terms such as Operator and Registry	Deployment/recovery, source ledger

For example, an arc42 quality scenario would say: "After loss of the application host, restore Mongo documents, media objects and Registry identity from a coherent backup, then verify an authenticated product path and a Registry receipt." Its recovery-time target and acceptance result remain unset until an owner supplies and tests them. This turns a vague "reliable" claim into a reviewable question without inventing an SLA.

TOGAF ADM: what a scoped change would ask

TOGAF's Architecture Development Method is iterative, with requirements managed across phases. Apply it here to a hypothetical question: should MyScoutee promote the KVM split beyond its lab? The current baseline is the published Compose product plus a distinct KVM lab; the target is undecided. The table is a small-scope example, not a mandate to run a full enterprise program.

ADM work	MyScoutee question and possible output	Gate before claiming a target
Preliminary and A: scope/vision	Who owns product, runtime and release decisions? Why consider a split: capacity, isolation or operating cost?	Named sponsor, constraints and measurable benefit
B: business architecture	Which member and Operator journeys must remain intact? What support and on-call burden is acceptable?	Product outcome and responsibility map
C: application/data	Which Java domain contracts move between processes, and who owns REAL/DEMO state, outboxes and Mongo writes?	API/data ownership and compatibility evidence
D: technology	Compare Netcup Compose, KVM/K3s and any cloud option on capacity, observability, recovery and invoice drivers	Verified runtime topology and cost assumptions
E/F: options/migration	Keep Compose, promote selected assemblies, or defer; sequence data migration, rollback and dual-form testing	Chosen option, migration plan and explicit stop rule
G/H: governance/change	Check release qualification, restore proof and drift after deployment; revisit when evidence changes	Accepted release evidence and an owner for architecture updates

The Open Group's ADM phase descriptions support this mapping. Existing cost, cloud and evidence chapters supply inputs, not a completed Phase E/F decision.

ADR: one decision, when it is actually made

For the same KVM question, a proposed ADR would record: Context - the Compose release and KVM lab have distinct qualification; Options - retain Compose, promote selected assemblies, or defer; Decision - pending the named owner and evidence; Consequences - deployment, cost, data and rollback responsibilities for the selected option; Confirmation - source changes plus QA, release and restore checks. The accepted record should link the exact revisions and supersede earlier records explicitly. Until then, this is a decision prompt, not a deployment plan. The ADR template reference describes the short context/decision/consequences pattern.

The smallest useful combination here is a current C4 context/container map, the existing arc42-like chapter structure, and an ADR for each material accepted choice. TOGAF ADM becomes useful when a change crosses business, application/data and technology ownership and needs a migration decision. Keep every view tied to the source ledger and label present, lab and proposed states separately.

7. Firebase in the architecture

Firebase is a configurable external provider for authentication and push messaging. It is not the product database, media store, payment system or source of application authorization. See security and integrations.

Capabilities and flow

Capability	Implemented path	Product boundary
Sign-in	Angular auth service uses the web project; Java verifies the credential and resolves or links a user	Java session and domain policy determine roles and access
Push	Angular obtains a permitted web-push token; Java Messaging sends through notification delivery	Device, outbox and recipient rules remain product-owned
Operator setup	Public web settings are served after staged settings pass tests and are activated	Auth and messaging tests must match the active revision

`DeploymentFirebaseRuntime` keeps separate Admin app identities for authentication and messaging, rejects live use in Explore/demo, and refreshes them after an activated revision changes. Startup `FIREBASE_ENABLED` is supported; the installed package defaults to `false` until Operator activation. Frontend flags alone do not activate the backend.

Configuration inventory

Surface	Settings or material	Treatment
Browser web configuration	`apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId`, `appId`; optional `measurementId`; web-push `vapidKey`	Public client metadata, served only for the configured project; the API key is not a server credential
Frontend feature switches	`firebaseLoginEnabled`, `firebaseMessagingEnabled` per Angular environment	UI capability switches; they do not activate backend credentials or grant access
Server startup/packaging	`FIREBASE_ENABLED`, `FIREBASE_SERVICE_ACCOUNT_PATH`, mounted service-account file	Server-only deployment inputs; never put the private service-account JSON in this guide or browser bundle
Staged deployment configuration	Project ID, revision, active flag, public client fields, encrypted authentication and messaging credential envelopes	Operator-owned state; both credential types and test revisions are required for runtime readiness
Legacy `fcm` properties	`url`, `apiKey` in `FirebaseProperties`	Separate configuration binding in source; do not confuse it with the public Firebase web API key or activated Admin credentials

`runtimeReady` requires an active revision, project ID, both encrypted credential envelopes, API key/auth domain/sender ID/app ID/VAPID key, and auth and messaging tests for that revision. `storageBucket` and `measurementId` are modeled but not required by this predicate. Example JSON files contain shapes, not deployable credentials.

Trust and evidence boundary

The product still checks identity linkage, sessions, REAL/DEMO context, roles, group scope and notification rules. Activation tests do not prove every browser/device delivery. Rotate secrets through the application owner's procedure, preserving encrypted settings and the server credential path.

Software delivery methods: from sequential plans to agile delivery

Status: general methodology reference and proposed RUP-to-agile transformation model, not a statement that MyScoutee currently uses any named framework. The approaches below overlap in time and solve different problems; the comparison is a map of practices and trade-offs, not a claim that one method historically replaced the next.

What is being compared?

A lifecycle model says when requirements, design, implementation, testing and release receive attention. A process framework assigns work, roles, decisions and review points. A team workflow controls how work enters and leaves the team. Engineering practices keep changes safe, while operations practices make releases and recovery reliable. A team can, for example, plan an external release in stages, manage day-to-day work with Kanban, use XP testing/refactoring practices and keep RUP-style risk evidence. These are choices at different layers, not mutually exclusive badges.

Waterfall-style planning: useful sequence, dangerous one-pass execution

The familiar waterfall picture places requirements, design, coding, testing and operations in a mostly sequential plan. It makes dependencies and external approval points visible, but a single pass with testing near the end exposes integration and performance surprises when change is expensive. The frequently cited 1970 Royce paper drew this sequence and then warned that its simple implementation was risky; it proposed feedback, early design/prototyping and customer involvement. Therefore, do not attribute a no-feedback waterfall prescription to Royce. Stage gates may still be needed for contracts, safety or release approval; verification should occur inside stages as well as at the gate.

The important correction: RUP is already iterative

RUP has four phases - Inception, Elaboration, Construction and Transition - and each phase can contain iterations. IBM describes an iteration as a complete development loop producing an executable increment. The shift toward agile is therefore not simply “start iterating.” It is a shift in *how small and valuable increments are selected, who can change priorities, how often users see them, and how much governance is needed for a decision*. IBM RUP best practices, IBM project planning.

The two stages below are a recommended transition design, not a claimed historical sequence or a formal standard. OpenUP and Agile Unified Process (AUP) are related lightweight Unified Process examples, not mandatory consecutive certifications. OpenUP is archived, so use its ideas as a reference rather than assuming active tool support. Eclipse OpenUP overview, Eclipse project status, Scott Ambler on AUP.

What RUP carried forward and what agile changed

This matrix compares the same concerns across approaches. The two middle transition stages are practical tailoring options described below. They are not named industry standards or a required route from RUP to Scrum.



Concern	Sequential stage plan	RUP baseline	Stage 1: tailored RUP	Stage 2: lean Unified Process	Agile product delivery
Main control	Phase completion and approval	Phase objectives, risk and milestone evidence	Same phase intent, shorter demonstrable iterations	Minimum useful roles/artifacts and visible risks	Product goal, ordered value, inspection and adaptation
Work selection	Requirements baseline and change requests	Use cases and risks in iteration plan	Slice use cases into end-to-end outcomes	Small, revisable work list and visible risks	Ordered backlog or flow; select a near-term goal
Architecture	Design before most construction	Elaborate high risks before scaling construction	Prove one risky vertical slice	Just enough architecture for current risk	Continuous design; retain explicit system-risk decisions
Testing	Often concentrated late in a strict one-pass plan	Test within iterations and assess milestones	Tested executable slice each iteration	Frequent integration and checks	Tested usable increments; fast feedback
Review	Approval at stage boundaries	Milestone and iteration assessments	Demo/test each iteration	Frequent feedback and process adjustment	Review user outcomes and adapt priorities
Evidence	Phase deliverables and traceability	Tailored artifacts for risk and governance	Keep proof of risk, compliance and recovery	Fit-for-purpose work products	Keep evidence that changes decisions and supports safety
Release	Final transition after earlier stages	Transition phase organizes rollout	Release-capable output after slices where feasible	Explicit release planning	Release when ready; review is not a release gate
Decision rights	Often centralized change approval	Often formal project/phase governance	Clarify scope and quality/risk owners	Routine decisions closer to the team	Product and delivery accountabilities within guardrails

The RUP column describes the process framework, not a judgment that all RUP teams create too much documentation. The agile column draws on the Agile Manifesto and principles and uses Scrum as one concrete example, not as the only agile method.

Stage 1 - make RUP increments genuinely inspectable

Keep the four phase names and risk focus if they help stakeholders. Change the iteration contract: every iteration ends with a running, tested vertical slice and a stakeholder decision. Plan only the next small increment in detail. Track a short risk list with evidence that each risk is reduced or still open. Treat a milestone as a decision on *actual software and remaining risk*, not a document-completion percentage. IBM's own planning guidance expects executable, demonstrable and testable iteration outputs, so this stage is a stricter application and tailoring of RUP, not a replacement. IBM planning paper.

Exit signal: stakeholders can inspect a small increment, see passed and failed checks, and reorder the next work without reopening a large phase plan. Keep safety, migration and release evidence when required.

Stage 2 - make the Unified Process lean

OpenUP preserves Inception, Elaboration, Construction and Transition while emphasizing collaboration, early architecture/risk work and iterative increments. It is intentionally low ceremony and extensible. AUP was Scott Ambler's earlier simplification of RUP for software development. These are alternative references for removing work products that do not change decisions. Choose a minimal set: product vision, ordered work, architecture decisions, tested increments, deployment/recovery evidence and open risks. Avoid copying a full process library into every issue. Eclipse lifecycle, Eclipse minimal process, Ambler's AUP context.

Exit signal: architecture and compliance evidence remains findable, while the team can change scope and improve its process without a separate process-change project.

Agile product delivery - change the operating model

Adopt the Manifesto's preference for working software, customer collaboration and responding to change. In Scrum terms, an accountable Product Owner orders a Product Backlog, a cross-functional team creates a usable Increment in a Sprint, and stakeholders inspect and adapt. The Sprint Review is not a gate to releasing value. The Definition of Done makes quality explicit. The organization must also change funding, decision rights and feedback access; renamed meetings alone do not create agility. Agile Manifesto, 12 principles, Scrum Guide.

Exit signal: priority changes are based on user/product evidence; teams can deliver a usable, verified increment frequently; risks and constraints are visible without waiting for a large phase boundary.

Related methods operate at different layers

Lens	Scrum	Kanban	Extreme Programming (XP)	DevOps / continuous delivery
Primary question	What valuable increment will meet the next Sprint Goal?	How does work flow predictably from start to finish?	How can developers change code safely and frequently?	Can a verified change reach users and be recovered safely?
Control loop	Sprint, Review and Retrospective with a Product Goal and Definition of Done	Explicit workflow, WIP control and flow measures	Short releases, feedback, testing, refactoring and integration	Automated build/test/deployment, observability and recovery feedback
What it does not supply alone	A mandated coding or deployment practice	A product strategy or code-quality method	Organization-wide funding and governance	The right product goal or user demand

Scrum is a lightweight framework for complex work; Kanban specifies flow control and measures such as WIP, cycle time and throughput; XP is unusually explicit about engineering practices; DORA's continuous-delivery guidance focuses on keeping software deployable and making releases low risk. A team can combine compatible practices, but it must name who owns product priority, code quality, release authority and operational recovery.

Where agile practice breaks

Agile's short feedback loop depends on real decisions and a working technical foundation. The failure is usually in those conditions, not in the mere presence or absence of a Sprint label.

Missing condition	Observable failure	Required correction
User access or empowered product decisions	Backlog items are completed without learning whether they helped anyone	Name a decision owner and inspect a user or operational outcome
Automated tests, integration and sustainable design	Each short cycle adds defects and makes the next cycle slower	Protect a usable Definition of Done; invest in tests, refactoring and CI
Architecture, security and recovery evidence	Local increments pass while cross-system risk accumulates	Keep risk-driven design, traceability and release/restore proof proportional to consequence
Realistic capacity and external dependencies	Sprint commitments become a fixed-scope contract despite interruptions	Make work and waiting visible; forecast from actual flow and keep release gates explicit

Scrum itself calls the framework incomplete and requires transparency, inspection and adaptation; its Definition of Done cannot compensate for an absent test and release system. The Agile Manifesto principles explicitly require technical excellence, sustainable pace and frequent working software. For delivery claims, measure both speed and stability, rather than velocity or ceremony counts alone; DORA's metric history distinguishes throughput from instability. These sources support the conditions above; they do not prove that any particular MyScoutee team currently has these failures.

Clean Code: an engineering discipline, not a new lifecycle

Robert C. Martin's Clean Code (first edition 2008) organizes advice about naming, functions, error handling, tests and code review. Much of the underlying concern predates that label: Parnas's 1972 modularity paper addressed flexibility and comprehensibility, Fowler's Refactoring first appeared in 1999, and XP included testing, simple design, refactoring and continuous integration. The book popularized and recombined practices; it did not create the need for maintainable code. Repeating a rule under a new name does not make a process better. Judge a coding rule by whether it improves comprehension and safe change in the actual codebase, with tests and system boundaries intact.

Mapping the comparison to MyScoutee without inventing current policy

MyScoutee already has useful artifacts for such a transition: source-owned domain boundaries, a versioned QA matrix, release notes, product configuration, KVM qualification and an installer track. Those artifacts can support any chosen process. The repository inspection does not establish a Product Owner, Scrum team, sprint cadence or RUP phase governance. A real transformation decision should first record the current delivery cadence and decision rights, then choose a pilot product slice, its user feedback channel, quality gate and rollback evidence.

A useful pilot could select one small vertical change and measure: time from accepted idea to usable increment, time waiting for decisions, escaped defects, deployment recovery and whether the actual user goal was met. This is a measurement proposal, not a claim about current MyScoutee performance. Keep architecture, privacy, protocol and release evidence proportional to risk rather than dropping them to appear agile.

Marketing decisions and product evidence

Status: decision framework, not an adopted campaign plan. Marketing is in scope for this repository because it affects positioning, acquisition channels, product feedback and operating cost.

Current evidence and owner

The product has versioned pitch materials, articles and social posts, and a product statement: a completed, mutually welcomed real-world meeting. Retrospective marketing records describe past content work and the limitations of token/list-price estimates. Those records do not measure campaign reach, conversion, retention or revenue. The source ledger identifies the owners without duplicating copy or media.

Decision questions

Decision	Evidence to collect	Avoid claiming
Audience and positioning	Interviews, use-case demand, landing-page comprehension	That an attractive pitch proves product-market fit
Channel choice	Reach, qualified visits, consented sign-ups and cost per qualified action	That impressions alone equal useful acquisition
Creative format	Controlled variants, audience fit, production/review time	That a new asset improves conversion without a comparison
Product feedback	Onboarding completion, real-world meeting outcomes and reasons for drop-off	That usage volume alone proves the product goal
Budget allocation	Incremental spend, attribution uncertainty and opportunity cost	That a token-equivalent estimate is an actual invoice

A campaign proposal should state audience, promise, channel, hypothesis, budget cap, measurement window, privacy/consent treatment, stop rule and owner. Match the promise to product behavior and current qualification. Product-facing claims should cite an actual release or verified capability; a lab demonstration is not a production feature.

Example decision record shape

```
Decision: [one channel or message choice]
Status: proposed | trial | adopted | rejected
Audience and user problem: [...]
Current baseline and source: [...]
Hypothesis and success measure: [...]
Spend and production effort: [...]
Privacy, brand and product-claim review: [...]
Result, uncertainty and next decision: [...]
```

This is a reusable record format, not a request to start a campaign or publish content.

Reading the advertising numbers

Term	Practical meaning
Impression / reach	A display / the platform's estimate of distinct accounts exposed. Neither is a website visit.
Click / landing-page view (LPV)	An ad interaction / a platform-reported arrival after the destination loads. Compare both with the site's successful page requests; reporting windows and definitions can differ.
Lead / qualified lead	A person who expresses interest / one who also meets the campaign's stated fit and consent criteria. A submitted form is not yet a MyScoutee meeting.
Conversion	The action chosen for measurement, such as a consented sign-up or operator enquiry. Specify the event before comparing campaigns.

Metric	Full name	Short reading
CPM	Cost per mille (1,000 impressions)	Ad spend ÷ impressions × 1,000; visibility or reach cost, not visit cost.
CPC	Cost per click	Ad spend ÷ counted clicks; specify which click type.
CPL	Cost per lead	Ad spend ÷ recorded leads; a qualified lead may cost more.
CPV	Cost per view	Ad spend ÷ eligible video views; a view threshold depends on the format.
CTR	Click-through rate	Counted clicks ÷ impressions; response to the ad, not site engagement.
CVR	Conversion rate	Defined conversions ÷ eligible ad interactions; name the action and denominator.
CPA	Cost per action	Ad spend ÷ defined conversions; a lead is only one possible action.
CAC	Customer acquisition cost	Total sales and marketing cost ÷ new customers; broader than ad spend.
ROAS	Return on ad spend	Attributed conversion value ÷ ad spend; depends on values and attribution.

These are reporting ratios; some names also denote bidding methods. They do not by themselves describe an invoice or prove incremental impact. Compare campaigns only after fixing the event definition, attribution window, currency and period. Google Ads glossary, Google's CPM definition, Google's CPV definition, Meta Audience Network glossary.

For MyScoutee, a useful sequence is exposure → loaded visit → meaningful page interaction → consented sign-up or qualified operator enquiry → completed, mutually welcomed meeting. Record the cost and drop-off at each step. Ad-platform attribution, browser analytics and server logs answer different questions; use one time zone, campaign ID and measurement window when reconciling them.

Choosing a channel

These are campaign hypotheses, not measured MyScoutee performance rankings. Facebook and Instagram share Meta advertising tools but should be reported by placement and creative. Targeting availability and delivery depend on campaign objective, region, account eligibility and platform policy; the auction can deliver differently even inside an eligible audience.

Decision	Facebook	Instagram	LinkedIn	YouTube
Best first hypothesis	Local pilot and community invitation	Visual discovery of the meeting concept	Operator and venue/partner enquiries	Product explanation and familiarity
Creative fit	Clear local event or meeting invitation	Short visual story or Reels demonstration	Concrete pilot offer and partner benefit	Short or longer demonstration video
Targeting lens	Geography; eligible broad or interest settings	Geography; eligible broad or interest settings	Company, industry, job title and eligible matched audiences	Eligible demographics, interests, in-market/custom segments and placements
Check beyond the platform metric	Loaded visit and qualified action, not feed engagement alone	In-app-browser visibility and meaningful action, not a tap alone	Qualified enquiry, not a form fill alone	Later site action, not a video view alone

Platform references: Meta auction and delivery and lead-generation formats; LinkedIn targeting and lead forms; Google audience segments and YouTube video objectives.

Do not upload member profiles, private affinity data or contact lists for ad targeting without a separate privacy and consent review. Keep channel tests small and comparable; no universal cost-per-lead or platform winner is established by the current evidence.

Historical Meta measurement: what the separate tool showed

An owner-supplied private Meta traffic review dated 21 October 2026 compared Meta screenshots, nginx requests and a temporary landing-page diagnostic recorder. The recorder counted visibility and input in Instagram's in-app browser; its data went to an nginx debug log, outside the Java/Mongo product audit. This was a bounded investigation. Only aggregate findings appear here.

On 20-21 October, the review estimated 204 visitors by distinct IP address (103 with a Meta click ID) and counted 464 successful landing-page requests (127 with a click ID). These units cannot be equated: IP counts misestimate people, requests do not prove viewing, and missing click IDs do not prove another source. Among 97 measured Instagram browser opens, 86 had no recorded input, 56 were visible for at most one second, and median visible time was 0.842 seconds. This raised a traffic-quality question, not proof of bots or fraud. In adjacent three-hour windows around Spain's removal from targeting, Instagram page requests fell from 49 to zero; the exact timing and cause remain unverified. Meta screenshots covered different periods and had inconsistent counts, so they cannot be reconciled by subtraction.

Check the quality of arrival before optimizing cheap clicks or platform-reported landing-page views. Temporary diagnostics need a bounded purpose and retention; routine decisions need privacy-reviewed aggregate events. The ad data do not measure gender dynamics. MyScoutee's purpose includes addressing unequal attention and filtering in online dating; the founder describes a path to mutually welcomed real-world meetings. Pew's U.S. survey reports different message-volume experiences for men and women. That context does not show this campaign discriminated or the product solved the problem.

Cost and efficiency decisions

Status: analytical framework; no current cloud bill, infrastructure price or savings claim is established here. Prices and provider terms change, so any concrete comparison needs dated quotes and measured workloads.

For the concrete provider comparison, see why Netcup was simpler first and the AWS, Azure, Google Cloud and OCI billing matrix.

What to count

A useful choice compares total cost for the same service level, not only a VM or database sticker price. Record workload volume and peaks, retention, backup/restore, egress, storage growth, operations time, security/qualification, deployment risk and migration effort. Separate one-time transition cost from recurring cost. Include developer-host and test-lab resources only when they are part of the decision. Keep the current product and experimental labs in different rows.

Candidate	Current evidence	Cost question
Packaged Compose on current host	Installed product runtime	What are measured host, storage, backup, support and availability costs?
Eight-service KVM/K3s laboratory	Separate deployment experiment	Does isolation/scaling justify more deployment and monitoring work at the same workload?
Big Data/Elastic labs	Synthetic or local experiments	Which real product question warrants additional data copies and retention?
Managed cloud or alternate graph store	Historical trial to locate; see technology trials	What measured capability, migration and operations benefit exceeds total cost?

Comparison protocol

1. Define the same workload, data size, concurrency, latency target, recovery target and privacy constraints for each candidate.
2. Measure the present baseline before proposing replacement. Use actual bills or dated provider quotes for external prices.
3. Model compute, storage, network egress, backups, licenses, support, human operations, migration and exit cost separately.
4. Test a bounded pilot with rollback and representative data; compare p50/p95 latency, throughput, failure recovery and total spend.
5. Record uncertainty, sensitivity to growth and the point at which the decision changes.

A cost reduction is supported only when the same functional, security and recovery requirements remain satisfied. A smaller monthly line item can be more expensive after migration and operational labor. This chapter authorizes no infrastructure change or purchasing decision.

Why Netcup was simpler first: cloud and edge cost anatomy

Status: dated decision analysis, 27 October 2026. This is not a current invoice or a commitment to migrate. Product facts come from the inspected MyScoutee source; provider billing terms come from linked official pages and must be rechecked for a purchase.

The actual starting point

MyScoutee's packaged application is a Docker Compose deployment with nginx, Java API, MongoDB, Redis, MinIO, MCP and isolated demo services on one host. Installation records confirm the product runs on Netcup. Deployment records place the authoritative A record in Netcup CCP CloudDNS and specify outbound DNS and HTTPS for Let's Encrypt certificate issuance. They do not establish the host's exact monthly invoice.

Netcup's current VPS product page advertises a fixed plan with CPU, RAM, SSD, included traffic and DDoS protection. That made a single-host, self-managed Compose installation a simpler initial purchasing and operations model: one primary server plan and one place to size the nine-container stack. It does not make the host highly available, eliminate backups or qualify the same workload at larger scale. A domain and any paid backup/add-on remain separate if the actual contract charges them.

Like-for-like comparison, before adding managed services

A Google Cloud comparison should first model one Compute Engine VM running the same Compose stack, not immediately assume GKE, managed databases and a global load balancer. Conversely, a single VM on Google Cloud does not gain high availability merely by changing provider. Google's Compute Engine pricing bills machine resources, while disk and snapshot pricing and network pricing cover separate usage. The exact bill depends on region, machine family, provisioned storage, egress, discounts and uptime.

Cost or capability	Netcup single VPS	Google Cloud single VM	Optional managed/edge addition
CPU and memory	Plan resources in one recurring VPS price	Compute Engine machine resource charge	Autoscaling or multiple instances add capacity and cost
Primary disk	Plan SSD capacity; extras may cost more	Provisioned persistent disk charged separately, including unused allocated space	Managed DB/storage charges and replication add lines
Backup	Design, storage and restore test still required; plan/add-on terms matter	Disk snapshots/storage and any restore traffic billed separately	Managed backup/retention may add charges
Internet traffic	VPS page advertises included traffic, subject to plan/usage terms	Outbound data transfer is a separate meter; inbound rules differ	CDN can reduce origin bytes but has its own price model
Public DNS	Existing domain record managed in Netcup CCP CloudDNS	Cloud DNS is optional if Netcup remains authoritative; if selected, zones and queries are billed	Cloudflare DNS is another optional authority

Cost or capability	Netcup single VPS	Google Cloud single VM	Optional managed/edge addition
Outbound DNS for provider calls	Host and containers need working name resolution and HTTPS egress	A Compute Engine VM has a default DNS resolver; paid Cloud DNS is not required merely to resolve Firebase endpoints	Custom forwarding, private zones or managed DNS policy are separate choices
Ingress and DDoS	Netcup VPS advertises DDoS protection; application still owns TLS/nginx policy	VM networking alone is not the same as a managed application edge	External load balancer, Cloud Armor, Cloudflare or CDN each have distinct scope/cost
Operations	Team patches host, containers, databases and recovery	Same if Compose stays on one VM	Managed services shift, but do not remove, integration/incident work

This table distinguishes a required billable component from an optional architectural choice. A Google Cloud VM can run nginx directly, and the domain can continue using Netcup DNS; a load balancer, Cloud DNS, Cloud Armor, GKE or Cloud SQL is not inherently required. If one of those is chosen, include its charge and the benefit it actually provides. Cloud Load Balancing pricing, Cloud Armor pricing.

For equivalent single-host options on the other major clouds, see the AWS, Azure, Google Cloud and OCI stack and billing comparison. Its egress example isolates one traffic meter; this Netcup analysis remains the comparison to the current host.

DNS: two different jobs

1. Authoritative DNS publishes where `myscoutee.com` points. The MyScoutee deployment reference records an A record in Netcup CCP CloudDNS. Google's Cloud DNS is a separately metered alternative: managed zones plus queries, with no free tier in its published price list. Cloudflare offers authoritative DNS on its free plan without moving hosting. None of these is a prerequisite to moving only compute.

2. Outbound recursive resolution lets the host resolve external names, for example certificate and provider endpoints. The requester recalls needing it on the Netcup host for Firebase token refresh. The inspected Netcup guide independently requires outbound DNS and HTTPS for Let's Encrypt, but does not preserve a Firebase-specific incident, the token type or the exact failing endpoint. Firebase's FCM server guidance describes rotating short-lived OAuth access tokens; that is a possible context, not proof of which refresh the recollection refers to. This DNS job is distinct from the public A record. A Compute Engine VM has a default resolver for internal and public names, so buying a separately billed managed Cloud DNS zone is not a prerequisite for resolving Firebase endpoints. Network egress and resolver policy must still work.

The historical Firebase point concerns the second job. The Netcup CCP CloudDNS A record documents the first job, and the two should not be combined into a single cloud DNS cost. No separate Netcup DNS fee is evidenced in the inspected deployment record; verify the actual domain contract before calling it free.

Where Cloudflare could help image browsing

The Java media endpoint gives public media a one-year public cache directive and private media `private, no-store` after an access-policy check. nginx serves static frontend assets and proxies `/media/public` to Java. This split matters more than the fact that both responses contain an image. Cloudflare's cache behavior respects `private`/`no-store`, and CDN setup caches eligible proxied static resources. DNS-only records do not route traffic through Cloudflare cache or HTTP DDoS filtering.

Path	Possible edge treatment	Required proof
Hashed frontend images/fonts	Cache through a proxied Cloudflare hostname; existing immutable/static headers are a good starting point	Correct versioning, purge/rollout and cache-hit checks
`/api/media/public?key=...`	Candidate for an explicit cache rule because the URL lacks a normal image extension; retain key/query separation and origin policy	No private key can reach the public route; no stale or cross-key image; measured hit ratio and origin egress
`/api/media/private?key=...`	Bypass shared cache and image transformation under the current user-specific access model	No shared-cache HIT, no user-to-user disclosure, preserved auth/session and `no-store`

Cloudflare Images transformations can resize/compress images served from an external origin. Its pricing meters unique transformations; storing images in Cloudflare Images adds storage/delivery meters. The Free tier currently includes 5,000 unique transformations per month, while the Paid tier has usage charges. Those products are optional: standard proxied CDN caching and DDoS protection can be evaluated first. Cloudflare warns that resizing authenticated images into a shared cache can expose user-specific content; that is why the current private route is excluded. Authenticated-origin guidance.

Cloudflare's standard DDoS protection is available on all plans, but it protects the traffic actually routed through its edge. It does not replace origin hardening, application authorization or a plan for direct-to-origin traffic. Cloudflare can sit in front of Netcup; using it does not require moving the backend to Google Cloud. Its free authoritative DNS is a separate decision from enabling proxied HTTP traffic.

Decision model

For a measured month, compare:

```
Netcup total = actual VPS + domain + backup/add-ons + team operations
Google single-VM total = compute + provisioned disks + snapshots + outbound traffic
                       + chosen DNS/LB/security/CDN + team operations + migration
Netcup + Cloudflare total = Netcup total + selected Cloudflare plan/features
                          - measured origin egress or latency benefit
```

The last subtraction is a benefit estimate, not automatically a cash saving: included Netcup traffic may make origin-byte reduction valuable for latency/capacity before it reduces the invoice. Measure public/private image request counts, bytes, cache-hit rate, p95 viewing latency, peak load, false positives, and DDoS/incident needs. Then price the same service and recovery level in each candidate. A pilot should leave the private media path uncached and verify public image invalidation and TLS/host routing before any DNS cutover.

Comparable cloud stacks and billing: AWS, Azure, Google Cloud and OCI

Status: decision analysis, 27 October 2026. This is a pricing snapshot and an illustrative workload, not a MyScoutee migration plan or a measured production bill. Recheck the linked provider rates, region, currency, tax and contract before purchasing.

One workload, four self-managed stacks

The like-for-like baseline is the packaged MyScoutee Compose application: nginx, Java API, MongoDB, Redis, MinIO, MCP and demo services on one Linux VM. Keep the same software, database ownership, backup policy and availability target on every provider. A managed database, Kubernetes cluster, CDN or load balancer would be a different architecture and needs its own comparison. The current Netcup setup remains the baseline in the Netcup analysis.

For a repeatable quote, ask each provider for a benchmark-matched x86 VM around 4 vCPU and 16 GiB RAM, 200 GiB of suitable persistent SSD, one public IPv4 address, 730 running hours, one month of measured backup retention and 1,000 GB of internet response traffic to European users. These are illustrative inputs, not measurements of MyScoutee's current demand. OCI x86 shapes count one OCPU as two vCPUs, so a first OCI quote would use two OCPUs and 16 GiB, then compare benchmark results rather than CPU labels alone.

Layer	AWS	Azure	Google Cloud	OCI
Same application	EC2 Linux + Compose	Linux VM + Compose	Compute Engine + Compose	Flexible x86 VM + Compose
Persistent data	EBS gp3 + snapshots	Managed Disk + snapshots/backup	Persistent Disk + snapshots	Block Volume + backups
Public entry	IPv4; Netcup DNS or Route 53; optional CloudFront/WAF	Public IP; Netcup DNS or Azure DNS; optional Front Door/WAF	External IPv4; Netcup DNS or Cloud DNS; optional CDN/Armor	Public IP; Netcup DNS or OCI DNS; optional load balancer/WAF
Cost signature	VM time, EBS capacity/performance, IPv4 and egress billed separately	VM, disk SKU, public IP, egress and backup; Azure Backup can add two meters	VM, disk, IPv4 and egress; Premium/Standard routing changes transfer rate	OCPU and memory billed separately; Block Volume performance units; 10 TB outbound allowance

The application's MongoDB, Redis and MinIO containers remain on that VM in this comparison. Provider object storage or a Mongo-compatible API is not assumed to be a drop-in replacement. External Firebase calls still require working outbound DNS and HTTPS; moving authoritative DNS to the VM provider is not required for token refresh. Cloudflare can be evaluated in front of any of these origins for the public image route, while the private route remains uncached, as described in the edge analysis.

Where the bill changes

Meter	AWS	Azure	Google Cloud	OCI
Compute	EC2 instance time, with Linux partial hours billed per second after a 60-second minimum	VM size and running time; any licensed image/support may add cost	vCPU and memory/machine charges by second after a one-minute minimum	Flexible x86 shape bills OCPU-hours and memory GB-hours; standard-flex billing pauses when stopped
Persistent disk	EBS gp3 provisioned GB; extra IOPS/throughput only above the included baseline	Managed Disk tier and capacity; Ultra/Premium v2 performance settings can add meters	Persistent Disk provisioned capacity; Hyperdisk can separately bill IOPS/throughput	Block Volume capacity and selected VPUs/performance
Backup	EBS snapshot storage and any chosen backup orchestration	Snapshot storage or Azure Backup protected instance plus backup storage	Snapshot storage, with location/restore transfer possible	Block Volume backup/storage terms from the OCI quote
Public IPv4	In-use and idle IPv4 addresses have an hourly charge	Static Public IP is a separate SKU; check the exact allocation and lifecycle	In-use and reserved external IPv4 have separate rates	Verify the selected public-IP and networking line in the regional quote rather than assuming parity
Internet response bytes	First 100 GB/month free across eligible AWS services/regions, then region-specific transfer-out tiers	First 100 GB/month free; Europe Premium Global Network's next tier is priced per GB	Premium Tier defaults to destination-based GiB tiers; Standard Tier has a separate 200 GiB/month allowance and different routing	First 10 TB/month of public outbound transfer free in the published price list; price above that depends on source geography
Authoritative public DNS	Route 53 hosted zone and public queries if selected	Azure DNS zone and queries if selected	Cloud DNS zone and queries if selected	OCI DNS terms if selected; retain Netcup authority for the baseline
Edge/security	CloudFront, WAF or Shield Advanced only when chosen; Shield Standard has no additional fee	Front Door, WAF or enhanced DDoS plan only when chosen	Cloud CDN, load balancer or Cloud Armor only when chosen	Load balancer/WAF only when chosen; Cloudflare remains provider-independent

The billing descriptions are drawn from the linked official AWS EC2, EBS and VPC pages; Azure VM, disk, backup and bandwidth pages; Google Compute, disk and network pages; and the OCI price list and compute FAQ. Do not equate a stopped VM with a zero bill: disks, backups and reserved/static IP resources can continue billing.

Egress-only example: 1,000 GB delivered in one month

This isolates internet response traffic from one EU VM to European users, without a CDN, load balancer, taxes, discounts or other usage consuming the free allowance. It excludes the VM, disk, backup, IP, support and operations, so the amounts below are not total hosting prices. Azure and AWS tariff units are GB; Google tariff units are GiB. For Google, 1,000 GB decimal is 931.323 GiB. The Google Premium and Standard paths have different network routing and must be evaluated separately.

Meter	AWS Frankfurt	Azure Europe Premium	Google Premium to Europe	Google Standard	OCI European origin
Free / first paid rate	100 GB account-wide / $0.09 per GB	100 GB / $0.087 per GB	1 GiB / $0.12 per GiB	200 GiB / $0.085 per GiB	10 TB / above-allowance rate varies
Charge for 1,000 GB	$81.00 (900 GB paid)	$78.30 (900 GB paid)	$111.64 (930.323 GiB paid)	$62.16 (731.323 GiB paid)	$0.00 (within allowance)

Sources for this calculation: AWS public Frankfurt AWSDataTransfer price list, SKU F62Y6TKR8NC6Q2ZB, published 16 September 2026, plus the AWS 100 GB allowance; Azure Europe bandwidth tariff; Google Premium and Standard tariffs; and the OCI networking tariff. The AWS price-list SKU is a machine-readable official feed; its current-version URL can change. All amounts are USD and rounded to cents.

The example makes the traffic effect visible: OCI's published allowance can remove this particular egress line, while Google Premium can make it larger than the same illustrative Azure/AWS transfer. It does not establish that OCI has the cheapest full MyScoutee deployment. VM performance, persistent storage, snapshots, public IP, taxes, support, availability and team familiarity still have to be priced and tested. The Oracle option is less familiar in the current MyScoutee evidence base, so a bounded x86 Compose and restore pilot would be needed before assigning operational savings.

Quote and decision rule

For each provider, save a dated regional quote with the selected VM SKU, CPU architecture, disk type/size/performance, backup retention, public IP lifecycle, 1,000 GB traffic path, any DNS/edge choices and support tier. Use the same measured MyScoutee load and recovery target. Compare the full monthly total and a growth case (for example 3,000 GB and doubled storage), then add migration and operations effort separately. Do not count promotional credits or an interruptible/spot VM as a production baseline unless the workload can tolerate their terms.

Technology trials and integration status

Status: evidence inventory and decision prompts. A nearby repository or prior experiment does not establish an application integration.

Technology	Current scope	Current interpretation
Kafka, ClickHouse, Spark, Flink and S3-compatible lake	Big Data lab	Runnable synthetic lab; not connected to product data
Elasticsearch, Kibana, Logstash, Beats and Fleet	Elastic lab	Runnable local CodeForge/telemetry lab; not a product SIEM/search deployment
TOTP/HOTP/WebAuthn factor service	2FA lab	Standalone service; MyScoutee login integration not present
Ansible and Puppet	Infrastructure lab	Synthetic nginx provisioning comparison; no backend provisioning
Google Cloud trial	Mentioned as historical context by the requester; no versioned trial report found in the inspected repository READMEs/guides	Evidence to recover before describing workload, result or spend
Neo4j trial	Mentioned as historical context by the requester; no versioned trial report found in the inspected repository READMEs/guides	Evidence to recover before claiming graph-model fit or replacement value

The requester also recalls contacting Google Support during an earlier Google-related effort; the support representative responded from the Philippines. No case record, date, diagnosis, recommendation or outcome was found in the inspected versioned guides. This is a historical recollection, not evidence of a Google Cloud product defect or a paid support contract.

For Google Cloud, recover the exact service (for example a VM, managed database or serverless product), date, region, workload, configuration, bill/quote, outcome and reason to continue or stop. For Neo4j, recover the graph model, dataset size, query patterns, measured comparison to the current Mongo/affinity path, licensing/deployment assumptions and data-sync implications. These are information needs, not assertions about what was previously tested.

Before promoting a trial

A decision record should name the product problem, current owner and baseline, representative workload, data classification, integration contract, cost model, security/operations owner, rollback, measured result and status (`idea`, `lab`, `pilot`, `adopted`, `rejected`). Link immutable source evidence. The current architecture remains the product baseline until an owner implements and qualifies a change.